



Dynplayer

✓ 속성 4개 더 보기

dynplayer.win (데스크탑에서 이용 권장)

서비스의 개요

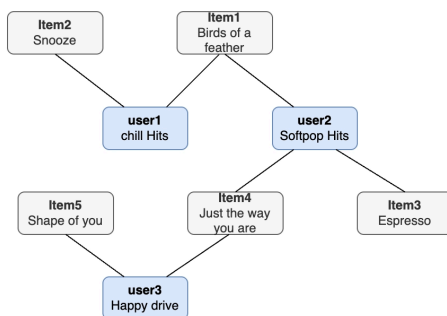
GOAL: 사용자의 engagement를 높일 수 있는 서비스를 만들자

- **pinterest**처럼 계속 탐색할 수 있는 UI를 구현합니다.
이 과정에서 리플 애니메이션과 빈티지한 커버 처리를 통해 계속해서 디깅하고 싶은 화면을 구성하였습니다.
- 실제 유저들의 음악청취 패턴을 파악하기 위해 **플레이리스트 데이터**를 실제 유저의 행동 로그라고 가정하였습니다.
- **recommendation:** simGCL 기반 트랙 임베딩 + 유저 시퀀스를 반영한 투타워 모델
- **keyword search:** clip 방식의 트랙임베딩과 키워드 임베딩의 alignment + 짧은 키워드 검색을 위한 전처리

Graph Embedding based Music Recommendation

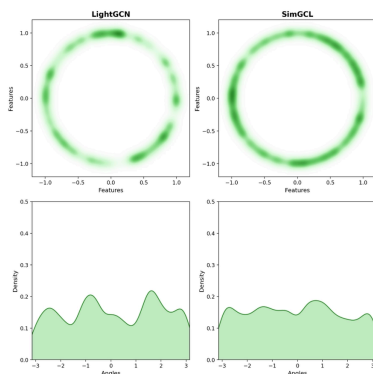
DATA

- Spotify 플레이리스트 데이터를 크롤링하여 구축
- 약 80K tracks, 40K playlists 규모
- 각 **playlist**를 하나의 **user sequence**로 가정하여 추천 문제를 구성
- user-item의 bipartite graph 생성하여 그래프 임베딩 학습에 사용



Graph Embedding

- **Item2Vec**
 - 플레이리스트 내 트랙 co-occurrence를 기반으로 트랙 임베딩을 학습 (**Skip-gram + negative sampling**)
 - 함께 등장하는 트랙은 가깝게, 무작위 트랙은 멀어지도록 학습
 - 대조학습 적용하였을 때 성능 크게 개선되었으나
 - 그래프 이웃 구조를 직접 활용하지 못한다는 한계 존재하여 그래프 임베딩 시도
- **LightGCN**
 - 사용자-아이템 이분 그래프에서 이웃 노드의 임베딩을 반복적으로 평균 전파(agggregation)하여 표현 학습
 - 유저-양성 아이템은 가깝게, 음성 아이템은 멀어지도록 최적화(BPR)
 - **Recall** 성능은 개선되었으나
 - 임베딩 공간이 uniform하지 않음
- **SimGCL**
 - **LightGCN** 구조에 랜덤 **perturbation**(노이즈)을 추가하여 서로 다른 view의 임베딩을 생성
 - 두 **view** 간 대조학습(InfoNCE)을 통해 같은 노드는 가깝게 다른 노드는 더 멀어지도록 학습
 - 임베딩 공간에서 uniformity 개선
 - $\mathcal{L}_{\text{uniform}} = \log \mathbb{E}_{x,y \sim p_{\text{data}}} \left[e^{-2\|f(x)-f(y)\|^2} \right]$
 - 결과적으로 더 **균형 잡힌 representation** 형성



t-SNE로 2차원 벡터 만든 후 KDE plot

플레이리스트에서 마지막 트랙을 제외한 나머지로 학습하고, 제외된 트랙을 맞추는 **Leave-One-Out(LOO)** 방식으로 평가

전체 성능 비교

Metric	Item2vec	item2vec(CL)	LightGCN	simGCL
Recall@10	0.0486	0.1386	0.1793	0.2291
Recall@20	0.0708	0.2103	0.2523	0.3082
NDCG@10	0.0262	0.0745	0.1006	0.1367
NDCG@20	0.0318	0.0926	0.1190	0.1565

Table1: Recall@10

Model	All	Top 10%	Bottom 10%
LightGCN	17.93	26.28	15.36
SimGCL	22.91	28.01	23.73

→ 특히 비인기곡(롱테일)에서 큰 개선

Table 2: Uniformity by Popularity

Model	All	Top 10%	Bottom 10%	Gap
LightGCN	-3.71	-3.29	-3.69	0.40
SimGCL	-3.85	-3.74	-3.83	0.09

→ SimGCL은 인기 아이템의 표현 분포를 더 균일하게 만들어 popularity bias를 완화하며, 그 결과 recall을 개선함

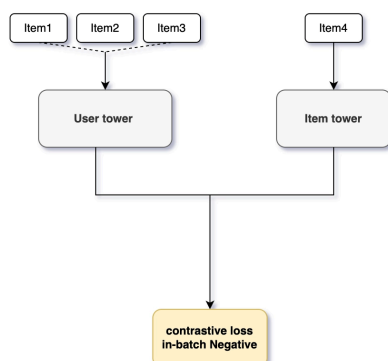
Table 3: Alignment by Popularity (between tracks in same playlists)

Model	All	Top 10% (Popular)	Bottom 10% (Unpopular)
LightGCN	0.8112	0.8457	0.4877
SimGCL	0.9209	0.9725	0.6922

그래프 모델은 유저 임베딩을 정적으로 학습하기 때문에, 현재 세션 기반의 동적 유저 표현이 불가능합니다.

이를 보완하기 위해 Two-Tower 모델을 도입하여 세션 내 트랙 임베딩으로부터 유저 임베딩을 실시간으로 생성합니다.

Two-Tower Model



- 플레이리스트는 순서보다는 맥락이 중요: bidirectional attention 이용
- 또한 실제 사용자의 시퀀스는 플레이리스트보다 짧을 것이기에 플레이리스트에서 시퀀스를 잘라서 학습에 이용

- **Architecture**
 - **User Tower**
 - 2-layer Transformer Encoder
 - 사용자 **track sequence** → **user embedding**
 - **Item Tower**
 - 트랙 임베딩을 **MLP**로 변환하여 같은 공간의 임베딩 생성
- **Training**
 - In-batch negative sampling + infoNCE
- **Serving**
 - item embedding을 **DB에 미리 저장**
 - **ANN** 검색으로 빠른 추천 수행
- **Evaluation (LOO)**

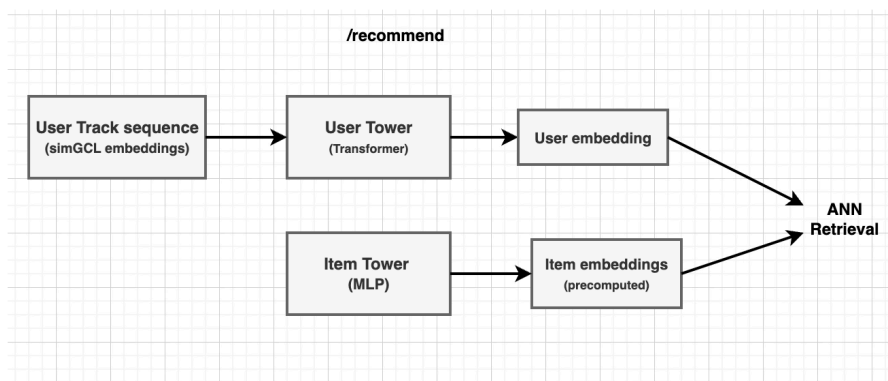
	Recall@10	Recall@20	NDCG@10	NDCG@20
Two-Tower + LightGCN	0.1604	0.2248	0.0935	0.1098
Two-Tower + SimGCL	0.1926	0.2683	0.1144	0.1335

그래프학습에서 우수했던 임베딩이 마찬가지로 더 좋은 성능을 보입니다.

HNSW 적용 결과 (m=16, ef_construction=64)

Metric	Recall
Recall@1	99.0%
Recall@5	99.0%
Recall@10	98.9%
Recall@20	98.85%

vector latency reduction: 86.8ms → 42.0ms



Keyword search

GOAL: 사용자가 검색한 무드/장르에 맞는 곡을 추천!

- Training

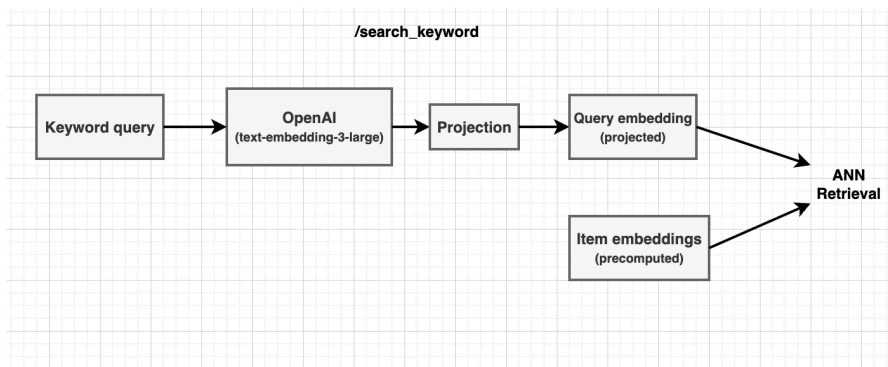
- 플레이리스트 제목은 대부분 길지만 (ex 눈 오는 거울에 듣는 감성힙합/발라드, 70s Hits - The Biggest Hits of the 70's)
- 유저의 검색은 짧은 키워드(ex 발라드, 크리스마스..) 위주가 될 것이라고 생각했습니다.
- GPT api를 통해 플레이리스트 제목에서 짧은 키워드를 추출하여 이를 학습에 사용하여 키워드 검색을 강화하였습니다.

70s Hits - The Biggest Hits of the 70's → ['70s', 'hits']

이후 3번 이상 등장한 2300여개의 태그를 학습에 사용하였습니다.

- inference

- 추론시에는 플레이리스트 임베딩과 같은 공간에 존재하는 트랙 임베딩에 프로젝션 모델을 사용합니다.
- projected embedding을 저장하여 검색 시간 단축합니다.
- ANN search(HNSW)
- 랭킹 모델(reranking)을 적용해 초기 검색 후보를 재정렬함으로써 검색 품질을 고도화



참고 논문

<https://arxiv.org/abs/1603.04259>

<https://arxiv.org/abs/2002.02126>

<https://arxiv.org/abs/2112.08679>

<https://arxiv.org/pdf/2103.00020>

